

Requirements

You need to implement a secure, production-grade backend system using **TypeScript**, **REST principles**, **Domain-Driven Design (DDD)**, and a **relational database** (e.g., PostgreSQL).

The system must be designed with a **security-first mindset**, strict access control, and robust domain modeling.

No open or bypassable endpoints are allowed.

Module 1: AI Chat Module

Functional Requirements

- Accepts a user question via a secured API endpoint.
- Returns a **mocked OpenAI response**.
- Stores the following in the database:
 - User question
 - AI answer
 - Token usage
 - Request metadata (timestamp, user reference)

Usage & Quota Management

- Track monthly usage per user.
- Each user receives **3 free messages per calendar month**.
- After free quota is exhausted, a **valid subscription bundle is required**. • Support **multiple active subscription bundles per user** with the following tiers:
 - Basic: 10 responses
 - Pro: 100 responses
 - Enterprise: unlimited responses
- Usage must be deducted from the bundle with the **latest remaining quota**.
- Free quota must **automatically reset on the 1st of each month**.
- If no quota is available, return **structured, typed error responses**.

Constraints

- Simulate OpenAI API response latency (mocked).
- Quota deduction must be:
 - Atomic
 - Safe under concurrent requests
 - Implemented using database transactions

Module 2: Subscription Bundle Module

Subscription Management

Allow users to:

- Create subscription bundles (Basic, Pro, Enterprise)
- Choose a billing cycle:
 - Monthly
 - Yearly
- Enable or disable auto-renew

Each subscription must include:

- maxMessages
- price
- startDate
- endDate
- renewalDate
- active / inactive status

Billing Simulation

- Automatically renew subscriptions if auto-renew is enabled.
- Simulate payment processing:
 - Payment failures occur randomly
 - On failure, mark the subscription as inactive
- Support subscription cancellation:
 - Ends the current billing cycle
 - Prevents future renewals
 - Preserves all historical usage data

Authentication & Authorization

- Integrate with an **external authentication provider** supporting OAuth2 / OpenID Connect.
- Support:
 - Email/password authentication
 - At least one OAuth provider
- Custom authentication implementations are not allowed.

Token & Access Control

- All endpoints must be protected.

- The backend must:
 - Verify access tokens server-side
 - Validate issuer, audience, and expiry
- Possession of an access token alone must **not** be sufficient to access APIs.
- Implement at least one additional security mechanism, such as:
 - Request signing
 - Proof-of-possession tokens
 - Nonce or timestamp-based validation
 - Session-bound token checks

Authorization

- Role-based access control:
 - user: access own chats and subscriptions
 - admin: system-wide access and analytics
- Authorization must be enforced at both:
 - Controller level
 - Domain policy level

Security Requirements

Mandatory Protections

- Secure HTTP headers
- Restricted CORS configuration
- Request size limits
- Strict content-type validation
- Global request timeout handling

Rate Limiting

- Per-IP rate limiting
- Per-user rate limiting
- Different limits for:
 - Authentication endpoints
 - Chat endpoints
 - Subscription endpoints

Input Validation & Sanitization

- Schema-based request validation
- Reject unknown or extra fields

- Sanitize against:
 - XSS
 - Injection attacks
- Prevent mass assignment vulnerabilities

Architecture

- Use **Clean Architecture (DDD-style)**.
- Enforce strict separation of layers:
 - domain/entities
 - domain/services
 - domain/policies
 - repositories
 - controllers
- Business logic must not depend on framework or transport layers.
- Organize modules independently:
 - chat/
 - subscriptions/

Technical Constraints

- TypeScript-first implementation (strict mode enabled).
- Relational database with migrations.
- Environment-based configuration using environment variables.
- ESLint and Prettier must be configured and enforced.

Observability & Error Handling

- Centralized error handling with structured JSON responses.
- Structured logging including:
 - Request ID
 - User ID
 - Response time
- Health check endpoint.
- Basic metrics endpoint (usage, subscriptions).

Testing Requirements

- Unit tests for:
 - Domain logic
 - Quota calculation
 - Subscription lifecycle

- Integration tests for:
 - Authenticated API access
 - Rate limiting
 - Security middleware behavior
- Authentication provider must be mocked in tests, not bypassed.

Submission Guidelines

Please follow the instructions below carefully:

- Create a public GitHub repository to manage your code.
 - Name the repository using your full name.
 - Add a README.md file that clearly explains:
 - Architecture decisions
 - Security model
 - Setup instructions
 - Upload the provided test PDF file to the repository as part of your submission. •
- Once you've completed the task, reply to this email with the link to your GitHub repository.

Deadline

You have **24 hours** to complete this task.

Submissions will not be entertained after the deadline.